LAPORAN PRAKTIKUM
POSTTEST 2
ALGORITMA PEMROGAMAN LANJUT

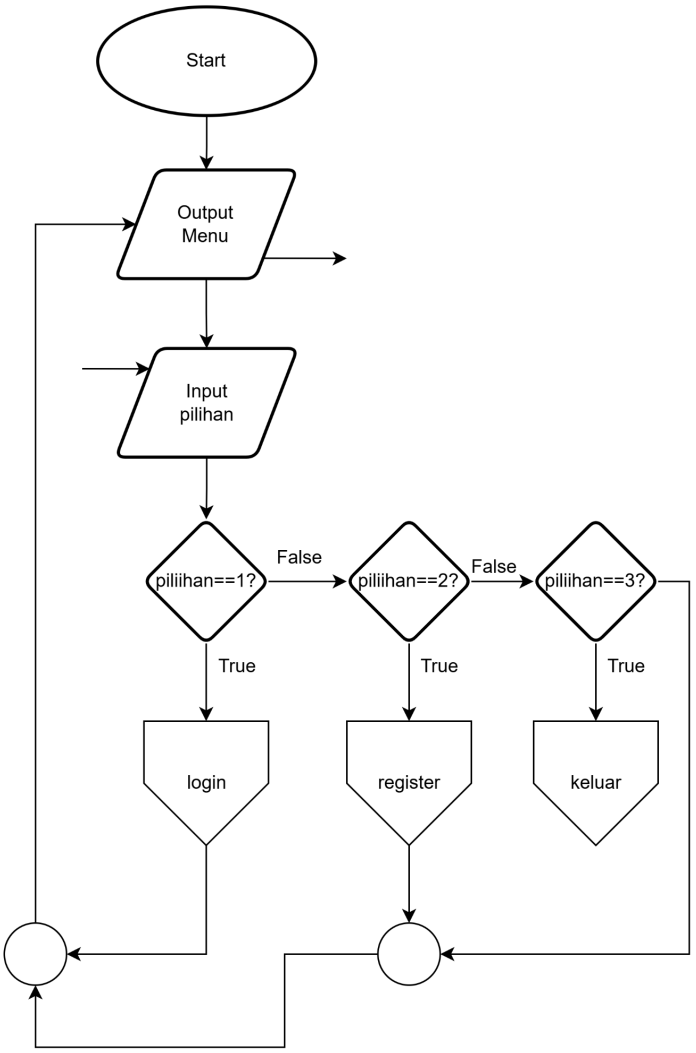
Disusun oleh:

Kikania Agarista (2509106086)

Kelas (B2'25)

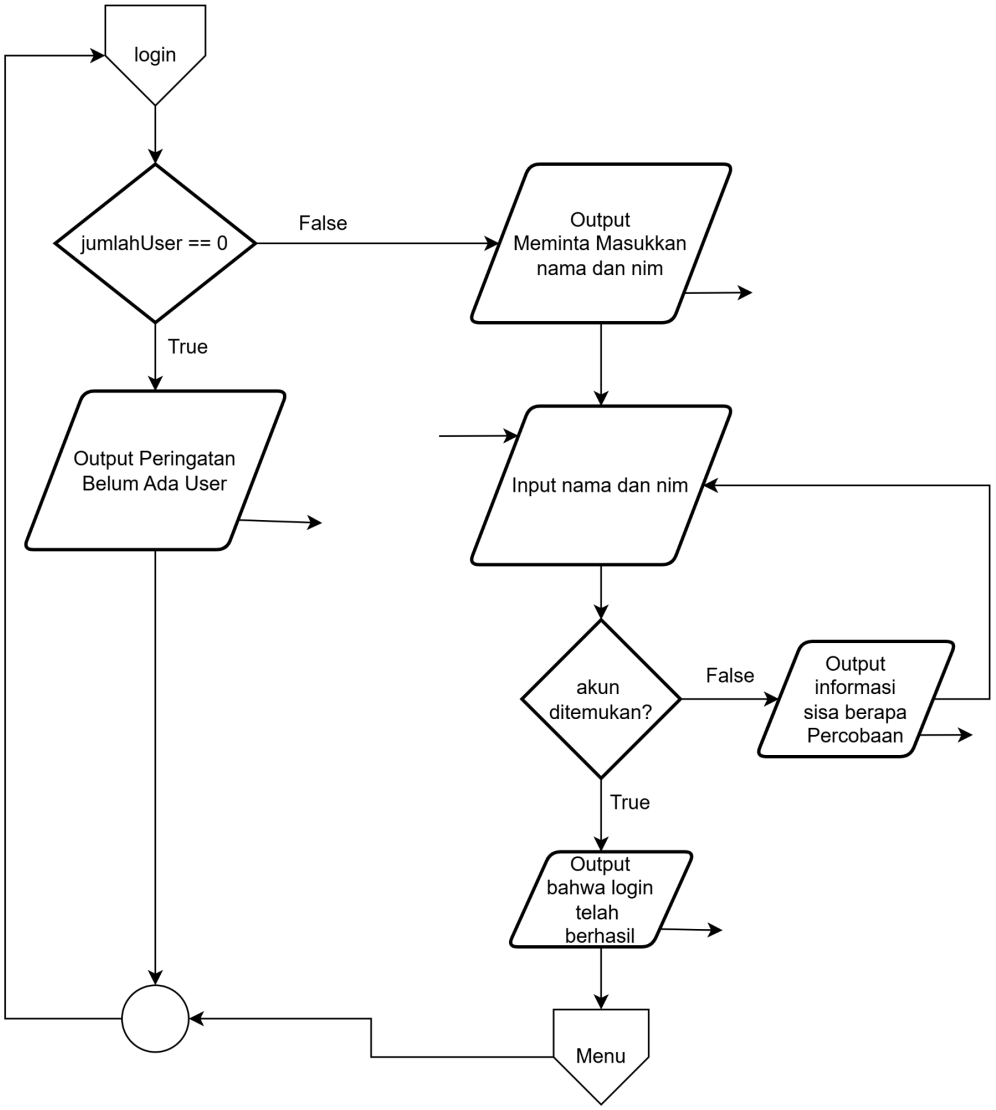
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
2026

1. Flowchart



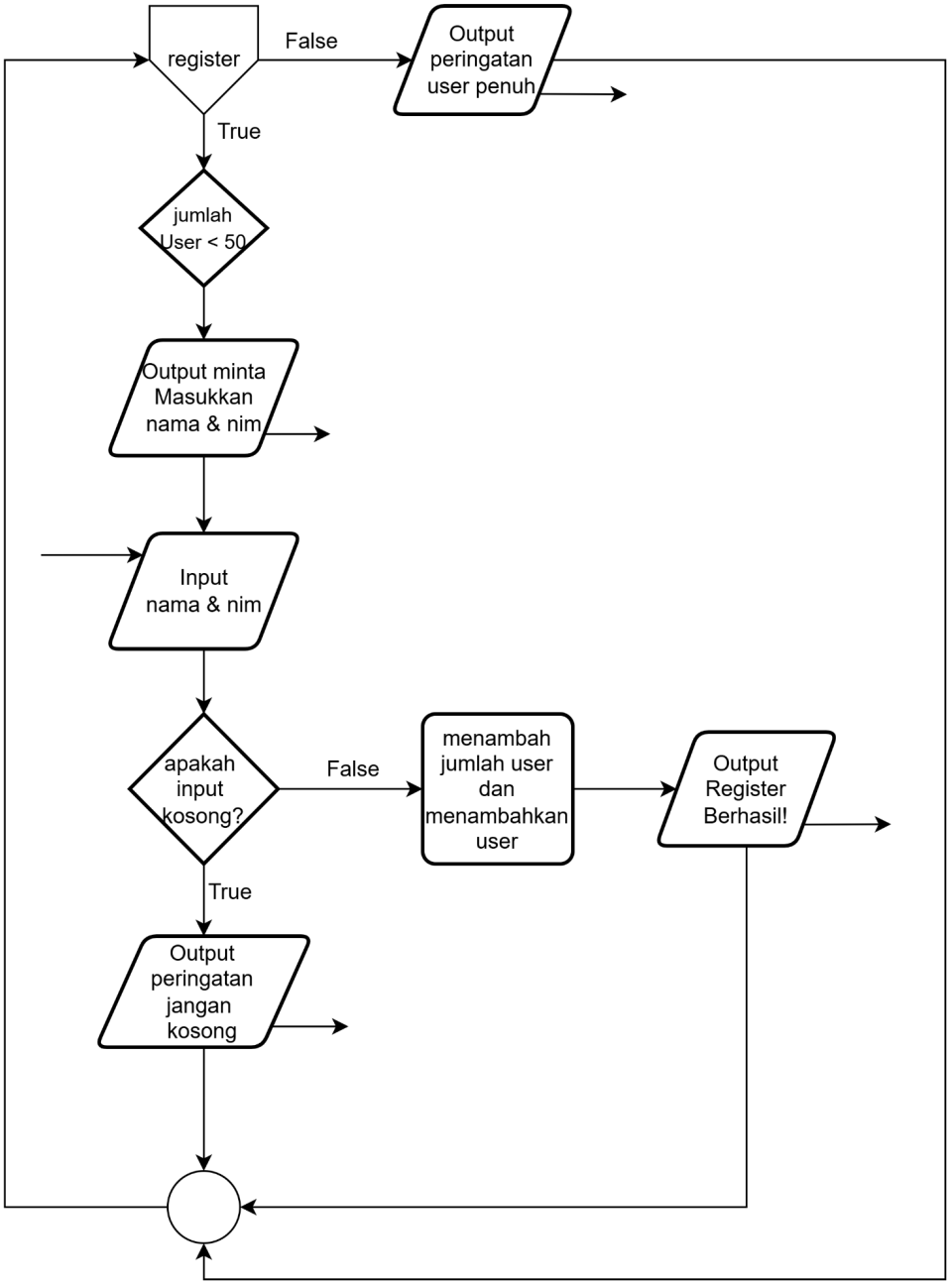
Gambar 1.1

Pada **Gambar 1.1**, diagram alir menjelaskan logika kerja dari sebuah sistem menu interaktif yang dimulai dengan proses inisialisasi (Start). Setelah itu, sistem akan menampilkan daftar pilihan kepada pengguna (Output Menu) dan menunggu masukan berupa angka pilihan (Input pilihan). Berdasarkan input tersebut, program akan melakukan percabangan kondisi (decision): jika nilai pilihan sama dengan 1, maka sistem akan menjalankan proses *login*; jika nilai pilihan sama dengan 2, sistem akan menjalankan proses *register*; dan jika nilai pilihan sama dengan 3, sistem akan menjalankan perintah *keluar*. Flowchart ini secara visual merepresentasikan struktur kendali seleksi (*if-else* atau *switch-case*) dalam navigasi menu utama aplikasi.



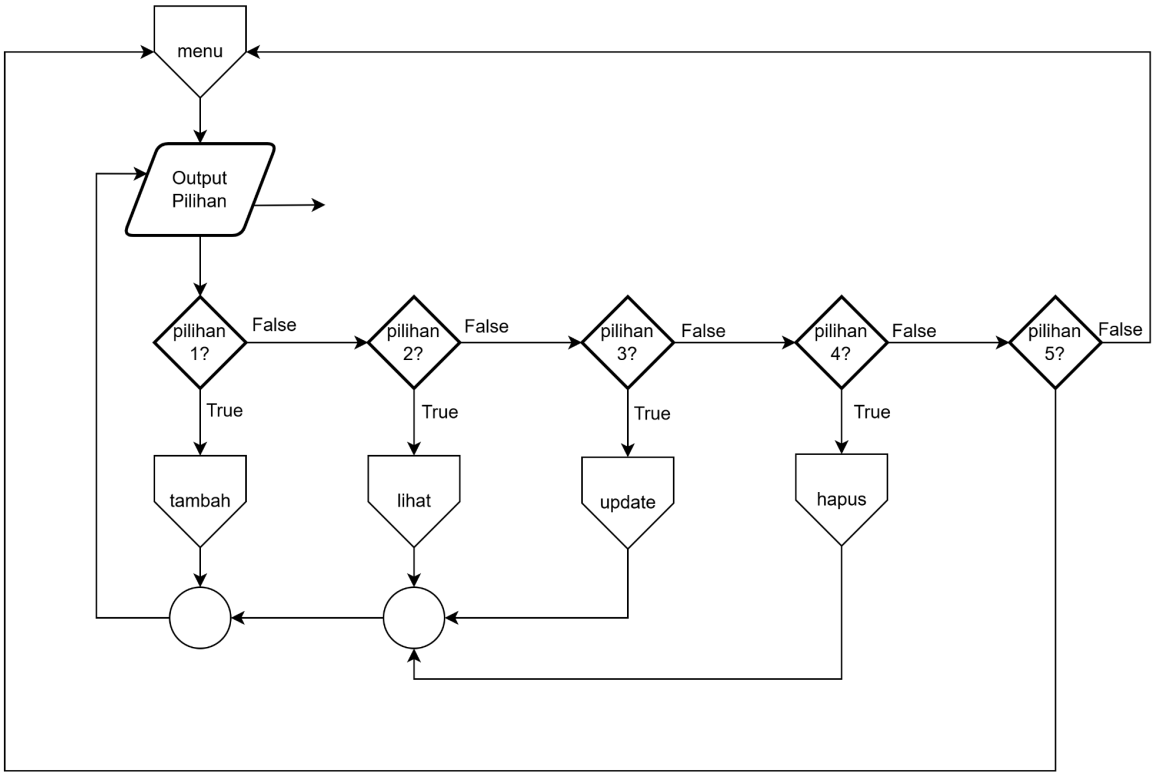
Gambar 1.2

Pada **Gambar 1.2**, diagram alir mendetailkan prosedur logika untuk fitur *login*. Proses dimulai dengan validasi awal untuk memeriksa apakah database pengguna kosong (**jumlahUser == 0**); jika kondisi ini terpenuhi, sistem akan langsung menampilkan peringatan bahwa belum ada user yang terdaftar. Apabila terdapat data user, alur berlanjut ke tahap input di mana sistem meminta pengguna memasukkan nama dan NIM. Selanjutnya, sistem melakukan pengecekan validitas data (**akun ditemukan?**); jika akun berhasil ditemukan, sistem akan menampilkan informasi mengenai sisa percobaan, memberikan notifikasi bahwa login berhasil, dan akhirnya mengarahkan pengguna kembali ke Menu utama.



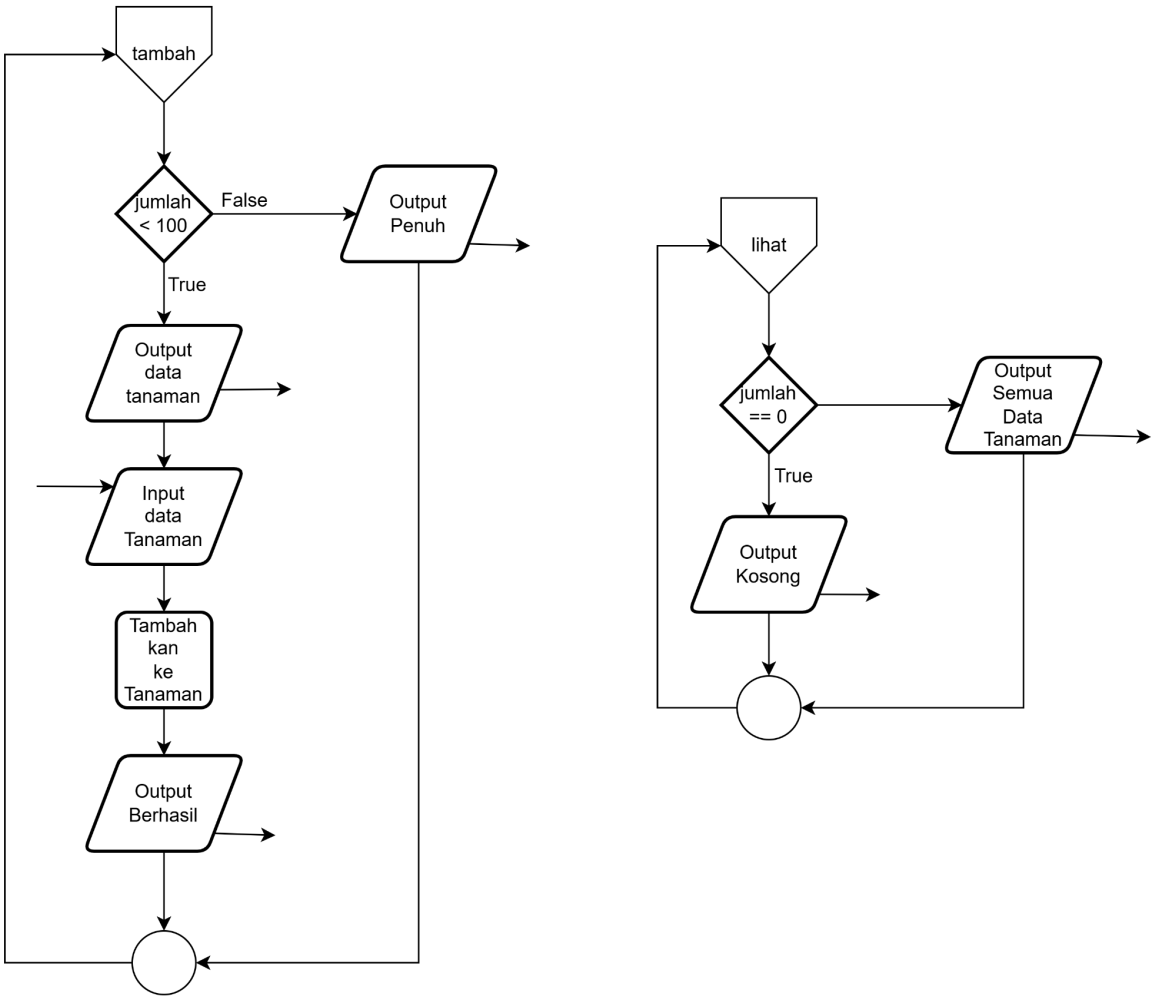
Gambar 1.3

Pada **Gambar 1.3**, diagram alir menguraikan logika prosedur untuk fitur registrasi atau pendaftaran akun baru. Proses diawali dengan pengecekan kapasitas penyimpanan pengguna melalui kondisi **jumlah User < 50**, jika kapasitas sudah penuh, sistem akan langsung menampilkan peringatan "user penuh". Apabila masih tersedia slot, sistem akan meminta dan menerima input berupa nama serta NIM dari pengguna. Selanjutnya, dilakukan validasi input untuk memastikan data tidak kosong (**apakah input kosong?**) jika input terdeteksi kosong, sistem akan memberikan peringatan, namun jika input valid, sistem akan memproses penambahan data pengguna, memperbarui jumlah user, dan menampilkan pesan konfirmasi "Register Berhasil!".



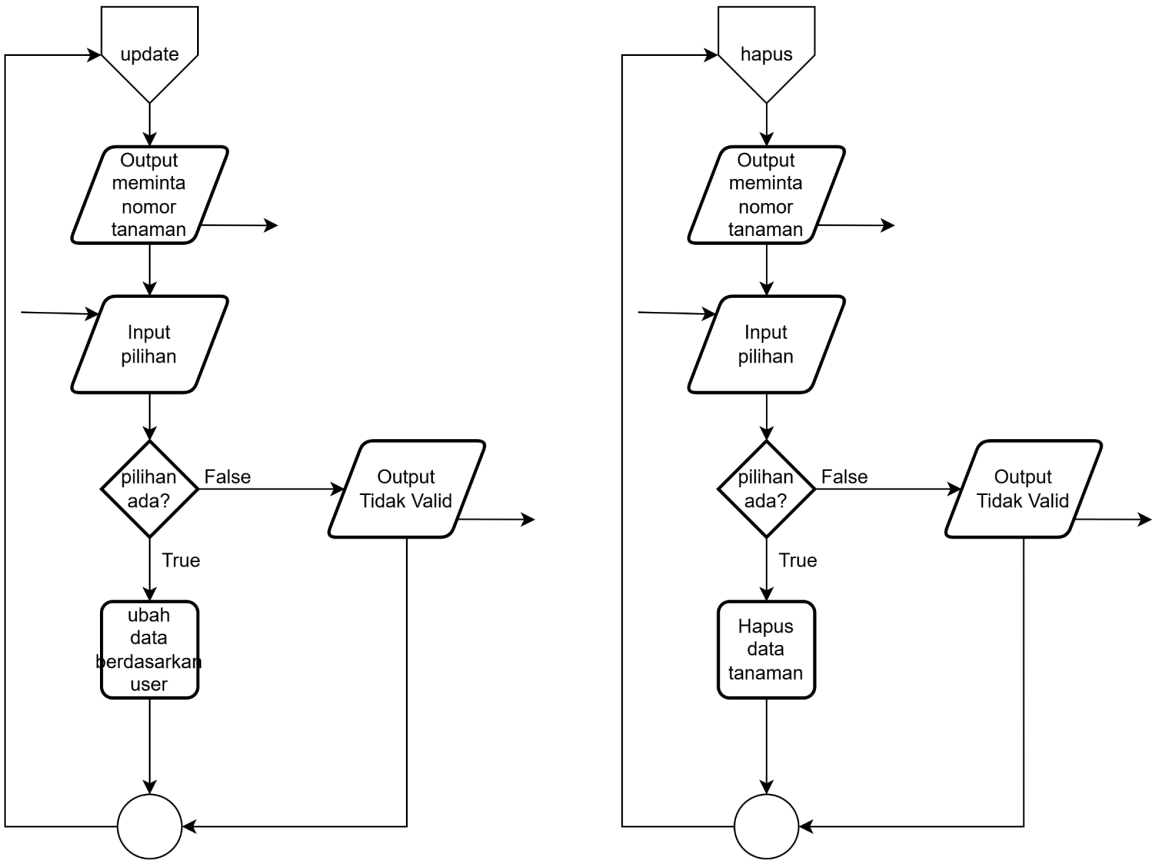
Gambar 1.4

Pada **Gambar 1.4**, diagram alir menjelaskan alur navigasi pada menu utama sistem yang diakses setelah proses login berhasil. Proses dimulai dengan menampilkan daftar opsi fitur (**Output Pilihan**) kepada pengguna. Berdasarkan input yang dimasukkan, sistem akan melakukan percabangan kondisi untuk mengeksekusi fungsi yang sesuai, jika pilihan adalah 1, maka akan masuk ke proses **tambah** data; jika pilihan 2, maka masuk ke proses **lihat** data; jika pilihan 3, maka masuk ke proses **update** data; dan jika pilihan 4, maka masuk ke proses **hapus** data. Terdapat pula opsi pilihan 5 yang tersedia dalam struktur percabangan tersebut, yang secara keseluruhan menggambarkan implementasi menu untuk operasi dasar data atau CRUD (*Create, Read, Update, Delete*).



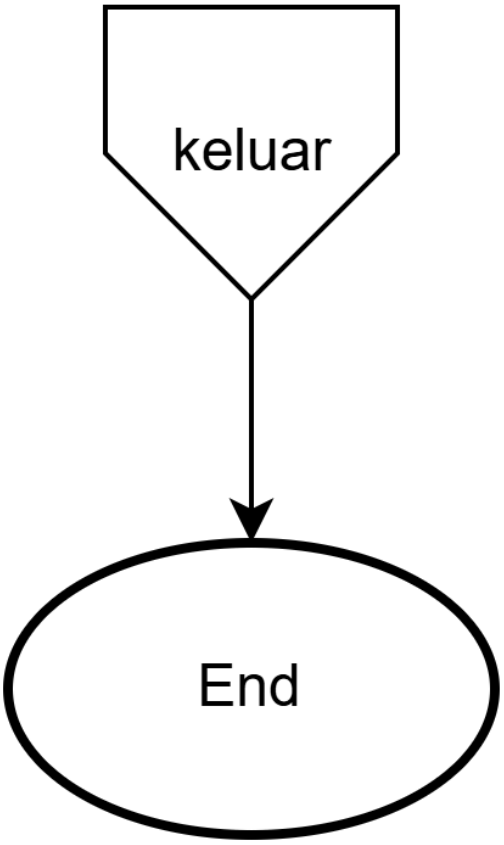
Gambar 1.5

Pada **Gambar 1.5**, diagram alir menjelaskan logika untuk dua operasi utama yaitu menambah ("tambah") dan melihat ("lihat") data. Pada alur "tambah", sistem melakukan validasi kapasitas dengan mengecek apakah **jumlah < 100**; jika kondisi salah (penuh), sistem akan menampilkan pesan "Output Penuh", sedangkan jika benar, sistem akan melanjutkan ke tahap menampilkan data tanaman, melakukan input data tanaman baru, memproses penambahan data tersebut ke dalam penyimpanan, dan memberikan konfirmasi "Output Berhasil". Secara paralel, pada alur "lihat", sistem mengecek apakah **jumlah == 0**; jika data kosong, sistem akan menampilkan pesan "Output Kosong", namun jika terdapat data, sistem akan menampilkan seluruh isi data tanaman ("Output Semua Data Tanaman").



Gambar 1.6

Pada **Gambar 1.6**, diagram alir menjelaskan logika prosedur untuk fungsi pembaruan ("update") dan penghapusan ("hapus") data tanaman. Kedua proses memiliki alur yang serupa, dimulai dengan sistem meminta nomor indeks tanaman melalui output permintaan, kemudian menerima input pilihan dari pengguna. Selanjutnya, sistem melakukan validasi keberadaan data tersebut dengan kondisi "pilihan ada?"; apabila data tidak ditemukan, sistem akan menampilkan pesan "Output Tidak Valid". Namun, jika data ditemukan, sistem akan melanjutkan ke aksi spesifik masing-masing fungsi, yaitu melakukan perubahan data sesuai masukan pengguna pada proses "ubah data berdasarkan user" untuk fitur update, atau langsung menghapus entri data tersebut pada proses "Hapus data tanaman" untuk fitur hapus.



Gambar 1.7

Pada **Gambar 1.7**, diagram alir menggambarkan tahap terminasi atau penghentian program yang diaktifkan ketika pengguna memilih opsi "keluar". Alur ini menunjukkan proses akhir yang langsung menghubungkan fungsi keluar menuju simbol terminator "End". Visualisasi ini menandakan bahwa seluruh rangkaian eksekusi sistem telah selesai dijalankan dan aplikasi akan berhenti beroperasi.

2. Deskripsi Singkat Progam

Program Sistem Pelacak Tanaman Hias ini merupakan aplikasi berbasis C++ yang dirancang khusus untuk mahasiswa baru IT sebagai implementasi praktis dari konsep MODUL 1 dan 2, dimana entitas User dan Tanaman dimodelkan menggunakan Nested Struct dan Array of Struct sesuai prinsip ERD sederhana. Aplikasi ini memungkinkan banyak pengguna untuk mendaftar dan login secara terpisah dengan batas maksimal tiga percobaan gagal, serta mengelola data tanaman pribadi melalui operasi CRUD (Create, Read, Update, Delete) yang disimpan sementara dalam memori tanpa menggunakan file eksternal atau fungsi tambahan. Seluruh logika program ditulis langsung dalam fungsi main menggunakan struktur kontrol dasar seperti perulangan dan percabangan, sehingga kode tetap sederhana,

mudah dipahami, dan fokus pada penguasaan manajemen data array serta struktur sebelum mempelajari konsep pemrograman yang lebih kompleks.

3. Source Code

```
#include <iostream>
#include <string>
using namespace std;


struct Tanaman {
    string namaTanaman;
    string jenisTanaman;
    int hariTerakhirSiram;
    bool statusHidup;
};

struct User {
    string namaUser;
    string nimUser;
    Tanaman daftarTanaman[100];
    int jumlahTanaman;
    bool statusAktif;
};

int main() {
    User dataUser[50];
    int jumlahUser = 0;
    int indexUserAktif = -1;
    int percobaanLogin = 0;
    string inputNama, inputNIM;
    bool programBerjalan = true;
    bool sudahLogin = false;


    while (programBerjalan) {
        if (!sudahLogin) {
            cout << "\n=== LOGIN ===" << endl;
            cout << "1. Login" << endl;
            cout << "2. Register" << endl;
            cout << "3. Keluar" << endl;
            cout << "Pilihan: ";
            int pilihan;
            cin >> pilihan;

            if (pilihan == 1) {
                if (jumlahUser == 0) {
                    cout << "Belum ada user. Silakan Register." << endl;
```

```
        } else {
            cin.ignore();
            cout << "Nama: ";
            getline(cin, inputNama);
            cout << "NIM: ";
            getline(cin, inputNIM);

            bool ketemu = false;

            for (int i = 0; i < jumlahUser; i++) {
                if (dataUser[i].statusAktif &&
                    dataUser[i].namaUser == inputNama &&
                    dataUser[i].nimUser == inputNIM) {

                    indexUserAktif = i;
                    sudahLogin = true;
                    percobaanLogin = 0;
                    ketemu = true;
                    cout << "Login Berhasil!" << endl;
                    break;
                }
            }

            if (!ketemu) {
                percobaanLogin++;
                cout << "Salah! Percobaan " << percobaanLogin << "/3" << endl;
                if (percobaanLogin >= 3) {
                    cout << "Program berhenti." << endl;
                    programBerjalan = false;
                }
            }
        }
    }
    else if (pilihan == 2) {
        if (jumlahUser < 50) {
            cin.ignore();
            cout << "Nama: ";
            getline(cin, dataUser[jumlahUser].namaUser);
            cout << "NIM: ";
            getline(cin, dataUser[jumlahUser].nimUser);

            if (!dataUser[jumlahUser].namaUser.empty() &&
                !dataUser[jumlahUser].nimUser.empty()) {

                dataUser[jumlahUser].jumlahTanaman = 0;
                dataUser[jumlahUser].statusAktif = true;
                jumlahUser++;
                cout << "Register Berhasil!" << endl;
            } else {
```

```
                cout << "Jangan kosong!" << endl;
            }
        } else {
            cout << "User penuh." << endl;
        }
    }
    else if (pilihan == 3) {
        programBerjalan = false;
    }
}

else {
    cout << "\n=== MENU - " << dataUser[indexUserAktif].namaUser << " ===" << endl;
    cout << "1. Tambah tanaman" << endl;
    cout << "2. Lihat tanaman" << endl;
    cout << "3. Update tanaman" << endl;
    cout << "4. Hapus tanaman" << endl;
    cout << "5. Logout" << endl;
    cout << "Pilihan: ";
    int pilihan;
    cin >> pilihan;

    if (pilihan == 1) {
        if (dataUser[indexUserAktif].jumlahTanaman < 100) {
            int index = dataUser[indexUserAktif].jumlahTanaman;
            cin.ignore();
            cout << "Nama Tanaman: ";
            getline(cin, dataUser[indexUserAktif].daftarTanaman[index].namaTanaman);
            cout << "Jenis: ";
            getline(cin, dataUser[indexUserAktif].daftarTanaman[index].jenisTanaman);
            cout << "Hari Siram (1-7): ";
            cin >> dataUser[indexUserAktif].daftarTanaman[index].hariTerakhirSiram;
            cout << "Hidup? (1=Ya, 0=Tidak): ";
            cin >> dataUser[indexUserAktif].daftarTanaman[index].statusHidup;

            dataUser[indexUserAktif].jumlahTanaman++;
            cout << "Berhasil!" << endl;
        } else {
            cout << "Penuh!" << endl;
        }
    }
    else if (pilihan == 2) {
        if (dataUser[indexUserAktif].jumlahTanaman == 0) {
            cout << "Kosong." << endl;
        } else {
            cout << "No|Nama|Jenis|Siram|Status" << endl;

            for (int i = 0; i < dataUser[indexUserAktif].jumlahTanaman; i++) {
                cout << i + 1 << "|"
```

```
                << dataUser[indexUserAktif].daftarTanaman[i].namaTanaman << "|"
                << dataUser[indexUserAktif].daftarTanaman[i].jenisTanaman << "|"
                << dataUser[indexUserAktif].daftarTanaman[i].hariTerakhirSiram << "|"
                << (dataUser[indexUserAktif].daftarTanaman[i].statusHidup ? "Hidup" : "Mati")
                << endl;
            }
        }
    }
    else if (pilihan == 3) {
        cout << "Nomor tanaman: ";
        cin >> pilihan;
        if (pilihan > 0 && pilihan <= dataUser[indexUserAktif].jumlahTanaman) {
            int index = pilihan - 1;
            cout << "Hari Siram Baru: ";
            cin >> dataUser[indexUserAktif].daftarTanaman[index].hariTerakhirSiram;
            cout << "Status Baru (1=Ya,0=Tidak): ";
            cin >> dataUser[indexUserAktif].daftarTanaman[index].statusHidup;
            cout << "Update OK!" << endl;
        } else {
            cout << "Tidak valid." << endl;
        }
    }
    else if (pilihan == 4) {
        cout << "Nomor tanaman: ";
        cin >> pilihan;
        if (pilihan > 0 && pilihan <= dataUser[indexUserAktif].jumlahTanaman) {
            cout << "Yakin? (1=Ya,0=Tidak): ";
            int konfirmasi;
            cin >> konfirmasi;
            if (konfirmasi == 1) {

                for (int i = pilihan - 1; i < dataUser[indexUserAktif].jumlahTanaman - 1; i++) {
                    dataUser[indexUserAktif].daftarTanaman[i] =
                    dataUser[indexUserAktif].daftarTanaman[i + 1];
                }
                dataUser[indexUserAktif].jumlahTanaman--;
                cout << "Hapus OK!" << endl;
            }
        } else {
            cout << "Tidak valid." << endl;
        }
    }
    else if (pilihan == 5) {
        sudahLogin = false;
        indexUserAktif = -1;
        cout << "Logout." << endl;
    }
  }
}
```

```
    cout << "\n=== SELESAI ===" << endl;
    return 0;
}
```

4. Hasil Output

```
PS C:\Praktikum-apl\post-test\post-test-apl-2> cd "c:\Praktikum-apl\post-test\post-test-apl-2\" ; if ($?)
{ g++ 2509106086-KikaniaAgarista-PT-2.cpp -o 2509106086-KikaniaAgarista-PT-2 } ; if ($?) { .\2509106086-Ki
kaniaAgarista-PT-2 }

=== LOGIN ===
1. Login
2. Register
3. Keluar
Pilihan:
```

Gambar 4.1 Output Login

```
=== LOGIN ===
1. Login
2. Register
3. Keluar
Pilihan: 1
Nama: Kikan
NIM: 086
Login Berhasil!
```

Gambar 4.2 Output Login

```
=== LOGIN ===
1. Login
2. Register
3. Keluar
Pilihan: 2
Nama: Kikan
NIM: 086
Register Berhasil!
```

Gambar 4.3 Output Register

```
=== MENU - Kikan ===
1. Tambah tanaman
2. Lihat tanaman
3. Update tanaman
4. Hapus tanaman
5. Logout
Pilihan:
```

Gambar 4.4 Output Menu Tanaman

```
=== MENU - Kikan ===
1. Tambah tanaman
2. Lihat tanaman
3. Update tanaman
4. Hapus tanaman
5. Logout
Pilihan: 1
Nama Tanaman: Anggrek
Jenis: Bunga
Hari Siram (1-7): 3
Hidup? (1=Ya, 0=Tidak): 1
Berhasil!
```

Gambar 4.5 Output Menu 1 Tambah Tanaman

```
=== MENU - Kikan ===
1. Tambah tanaman
2. Lihat tanaman
3. Update tanaman
4. Hapus tanaman
5. Logout
Pilihan: 2
No|Nama|Jenis|Siram|Status
1|Melati|Bunga|6|Hidup
2|Kasturi|Bungaa|5|Mati
3|wortel|sayuran|3|Hidup
4|mawar|bunga|2|Hidup
5|sawi|sayuran|5|Mati
6|lidah buaya|obat|7|Hidup
```

Gambar 4.6 Output Menu 2 Lihat Tanaman

```
=== MENU - Kikan ===
1. Tambah tanaman
2. Lihat tanaman
3. Update tanaman
4. Hapus tanaman
5. Logout
Pilihan: 3
Nomor tanaman: 3
Hari Siram Baru: 7
Status Baru (1=Ya,0=Tidak): 0
Update OK!
```

Gambar 4.7 Output Menu 3 Update Tanaman

```
=== MENU - Kikan ===
1. Tambah tanaman
2. Lihat tanaman
3. Update tanaman
4. Hapus tanaman
5. Logout
Pilihan: 2
No|Nama|Jenis|Siram|Status
1|Melati|Bunga|6|Hidup
2|Kasturi|Bungaa|5|Mati
3|wortel|sayuran|7|Mati
4|mawar|bunga|2|Hidup
5|sawi|sayuran|5|Mati
6|lidah buaya|obat|7|Hidup
```

Gambar 4.8 Output setelah Update

```
=== MENU - Kikan ===
1. Tambah tanaman
2. Lihat tanaman
3. Update tanaman
4. Hapus tanaman
5. Logout
Pilihan: 4
Nomor tanaman: 2
Yakin? (1=Ya,0=Tidak): 1
Hapus OK!
```

Gambar 4.9 Output Hapus Tanaman

```
=== MENU - Kikan ===
1. Tambah tanaman
2. Lihat tanaman
3. Update tanaman
4. Hapus tanaman
5. Logout
Pilihan: 2
No|Nama|Jenis|Siram|Status
1|Melati|Bunga|6|Hidup
2|wortel|sayuran|7|Mati
3|mawar|bunga|2|Hidup
4|sawi|sayuran|5|Mati
5|lidah buaya|obat|7|Hidup
```

Gambar 4.10 Output setelah Hapus Tanaman

```
=== MENU - Kikan ===
1. Tambah tanaman
2. Lihat tanaman
3. Update tanaman
4. Hapus tanaman
5. Logout
Pilihan: 5
Logout.

=== LOGIN ===
1. Login
2. Register
3. Keluar
Pilihan:
```

Gambar 4.11 Output LogOut

```
=== LOGIN ===
1. Login
2. Register
3. Keluar
Pilihan: 3

=== SELESAI ===
PS C:\Praktikum-apl\post-test\post-test-apl-2>
```

Gambar 4.12 Output Keluar Menu Login

5. Langkah-langkah GIT

```
PS C:\Praktikum-apl\post-test\post-test-apl-2> git add .
```

5.1 Git Add

```
PS C:\Praktikum-apl\post-test\post-test-apl-2> git commit -m "posttest2"
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

5.2 Git Commit

```
PS C:\Praktikum-apl\post-test\post-test-apl-2> git push
Everything up-to-date
```

5.3 Git Push